

R - Vektorindexelés és mátrixok: megoldókulcs

A magyarázatok azt emelik ki, amit az adott kifejezésből érdemes megérteni. A sample()-t használó példánál a konkrét számok véletlenek, ezért ott a működés a fontos.

Vektorok indexelése

```
proba=sample(1:100,30,replace=T)
```

Létrehoz egy 30 elemű vektort. Minden elem véletlenszerűen az 1 és 100 közötti egész számok közül kerül ki. A replace=T miatt ugyanaz a szám többször is szerepelhet. Minden futtatáskor más eredmény jöhet ki.

```
# Vektorok indexelése
```

```
proba[3]
```

A proba vektor 3. elemét választja ki. R-ben az indexelés 1-től indul.

```
proba[1]
```

A proba első elemét adja vissza.

```
proba[12]
```

A proba 12. elemét adja vissza.

```
proba[31]
```

A proba csak 30 elemű, ezért a 31. hely nem létezik. Pozitív, tartományon kívüli indexnél R NA értéket ad.

```
proba[1000000000000000]
```

Nagyon nagy, nem létező pozíciót kérünk. A lényeg ugyanaz: nincs ilyen elem; R egy hiányzó értékkel (NA) jelzi ezt, illetve a pontos viselkedés R-verziótól és méretkorlátoktól függhet.

```
proba[-1]
```

Minden elemet visszaad az első kivételével. A negatív index azt jelenti: ezt a pozíciót hagyd ki.

```
proba[-3]
```

Minden elemet visszaad a 3. kivételével.

```
proba[7:15]
```

A 7., 8., ..., 15. pozíciókon lévő elemeket választja ki.

```
proba[c(1,8,10,12,20,27)]
```

Egyszerre több, tetszőlegesen megadott pozíciót választ ki: az 1., 8., 10., 12., 20. és 27. elemet.

```
proba[seq(from=5,to=30,by=5)]
```

A seq() előállítja az 5, 10, 15, 20, 25, 30 indexeket, majd ezeken a helyeken lévő elemeket választja ki.

```
proba2=c(34,76,12,28,56,79)
```

Létrehoz egy 6 elemű numerikus vektort proba2 néven.

```
c(T,F,T,T,F,T)
```

Létrehoz egy 6 elemű logikai vektort. T=TRUE, F=FALSE.

```
proba2[c(T,F,T,T,F,T)]
```

Logikai indexelés. Csak azokat az elemeket tartja meg, amelyek pozícióján TRUE áll. Itt a 34, 12, 28 és 79 marad.

```
# (34,76,12,28,56,79)
```

```
# ( T, F, T, T, F, T)
```

```
# (34, ,12,28, ,79)
```

Feltételes kiválasztás vektorokból

```
# Hány szám van a proba vektorban, ami nagyobb, mint 70?
```

```
proba>70
```

A proba minden elemét összehasonlítja 70-nel. Ugyanolyan hosszú TRUE/FALSE vektort kapunk: TRUE ott, ahol az elem nagyobb 70-nél.

```
proba[proba>70]
```

A proba>70 logikai vektort indexként használja, ezért ténylegesen csak a 70-nél nagyobb számokat adja vissza.

```
length(proba[proba>70])
```

Előbb kiválasztja a 70-nél nagyobb elemeket, majd megszámlálja őket. Ez válaszolja meg, hány ilyen szám van.

```
proba[proba<20]
```

Kiválasztja a 20-nál kisebb elemeket.

```
proba[proba!=1]
```

Kiválaszt minden olyan elemet, amely nem egyenlő 1-gyel. A != jelentése: nem egyenlő.

```
proba[20<proba<50]
```

Ez NEM a matematikából ismert láncolt egyenlőtlenség. R előbb kiszámolja a 20<proba logikai vektort, majd azt hasonlítja 50-hez. Emiatt nem azt kapjuk, amit szeretnénk. Két feltételt & jellel kell összekötni.

```
proba[20<proba & proba<50]
```

Helyes megoldás a 20 és 50 közötti elemek kiválasztására. Egy elemnek egyszerre kell 20-nál nagyobblnak ÉS 50-nél kisebbnek lennie.

```
proba[proba%%2==1]
```

A %% az osztási maradékot adja. Ha 2-vel osztva a maradék 1, a szám páratlan. Ez tehát a páratlan elemeket választja ki.

```
proba[proba%%2==0]
```

A 2-vel maradék nélkül osztható, vagyis páros elemeket választja ki.

Sorrend, rendezés és szövegek

```
proba2[c(2,1,3,4,5,6)]
```

Új sorrendben adja vissza a proba2 elemeit: az első két elemet felcseréli, a többit változatlanul hagyja.

```
proba2[c(1,2,3,6,4,5)]
```

A megadott indexsorrend szerint rendezi át az elemeket: 1.,2.,3.,6.,4.,5.

```
sort(proba)
```

A proba elemeit növekvő sorrendbe rendezi.

```
sort(proba,decreasing=T)
```

A proba elemeit csökkenő sorrendbe rendezi. decreasing=TRUE azt jelenti: csökkenő sorrend.

```
View(state.x77)
```

Megnyitja R egyik beépített adathalmazát táblázatos nézetben. A View() megjelenítésre szolgál, nem számol új értéket.

```
order(proba2)
```

Nem magukat az értékeket rendezi, hanem azokat az indexeket adja vissza, amelyek sorrendjében a vektor növekvő lenne.

```
proba2[order(proba2)]
```

Az order() által kapott indexeket használja fel, ezért a proba2 elemei növekvő sorrendben jelennek meg.

```
proba[order(proba)]
```

A proba elemeit növekvő sorrendbe teszi az order() indexeivel. Eredménye ugyanaz a sorrend, mint sort(proba)-nál.

```
proba[order(proba,decreasing=T)]
```

Az order() csökkenő rendezési indexeit használja, így a proba elemei csökkenő sorrendben jelennek meg.

```
View(mtcars)
```

Megnyitja a beépített mtcars adathalmazt táblázatos nézetben.

```
proba3=c("alma","korte","narancs","meggy","dinnye","cseresznye")
```

Létrehoz egy szöveges, 6 elemű vektort.

```
sort(proba3)
```

A szövegeket rendezési sorrendbe teszi. A pontos sorrendet a gép nyelvi/locale beállítása is befolyásolhatja.

```
proba3>"l"
```

Minden szót összehasonlított az "l" szöveggel a rendezési szabályok szerint, és TRUE/FALSE értékeket ad. Szövegeknél ez nem hosszú, hanem rendezési sorrendet jelent.

```
proba3[proba3>"l"]
```

Csak azokat a szavakat választja ki, amelyek a használt rendezési szabály szerint az "l" után következnek.

```
proba3[proba3<="kutya"]
```

Azokat a szavakat választja ki, amelyek a rendezési sorrendben legfeljebb a "kutya" helyén vannak. A pontos eredmény locale-függő lehet.

```
substr(proba3,1,1)
```

Minden szó 1. karakterétől az 1. karakteréig kér részletet, vagyis minden szónak visszaadja az első betűjét.

```
proba3[substr(proba3,1,1)=="d"]
```

Előbb megnézi minden szó első betűjét, majd csak azokat tartja meg, amelyek d betűvel kezdődnek. Itt a dinnye kerül kiválasztásra.

Mátrixok létrehozása

```
# Mátrixok
```

```
# 1 2 3
```

```
# 6 4 9
```

```
# 1 9 4
```

```
# (1 6 1 / 2 4 9 / 3 9 4)
```

```
array(c(1,6,1,2,4,9,3,9,4),dim=c(3,3))
```

3x3-as tömböt/mátrixot készít a 9 számból. R alapértelmezésben oszloponként tölti fel: először az első oszlopot, majd a másodikat, végül a harmadikat.

```
array(1:40,dim=c(5,8))
```

5x8-as, azaz 40 helyes tömböt készít az 1:40 értékekből. Pontosan annyi adat van, ahány hely, ezért nincs ismétlés vagy elhagyás.

```
array(1:40,dim=c(5,6))
```

5x6=30 hely van, de 40 adatot adtunk. Az array() a szükséges 30 elemet használja; a fölösleges értékek nem kerülnek bele.

```
array(1:30,dim=c(5,8))
```

5x8=40 hely van, de csak 30 adat. R újrahasznosítja (recycling) az adatokat, hogy feltöltse a hiányzó helyeket.

```
matrix(1:40,nrow=5)
```

Mátrixot készít 5 sorral. Mivel 40 adat van, R kiszámolja, hogy 8 oszlop kell. Alapértelmezésben oszloponként tölti fel.

```
matrix(1:40,nrow=5,ncol=8)
```

Kifejezetten 5 soros és 8 oszlopos mátrixot hoz létre az 1:40 számokból.

```
matrix(1:40,ncol=8)
```

8 oszlopot kérünk; 40 adatból R 5 sort tud kialakítani.

```
matrix(1:40,nrow=5,byrow=T)
```

5 soros mátrixot készít, de byrow=TRUE miatt soronként tölti fel, nem oszloponként.

```
matrix(1:40,nrow=3)
```

3 sort kérünk 40 adatból. A 40 nem osztható pontosan 3-mal, ezért R-nek ki kell egészítenie/újrahasznosítania az adatokat; tipikusan figyelmeztetést is ad.

```
matrix(1:40,nrow=3,ncol=8)
```

$3 \times 8 = 24$ helyet kérünk 40 adatból. A megadott adatmennyiség és a mátrix mérete nem illeszkedik. R-verziótól függően figyelmeztetés vagy hiba is lehet; a tanulság, hogy a data hossza és a kért $nrow \times ncol$ méret legyen összeegyeztethető.

```
matrix(1:40,nrow=7,ncol=8)
```

$7 \times 8 = 56$ helyet kérünk, de csak 40 adat van. R az adatokat ismétli (recycling), hogy kitöltse a mátrixot; az eltérő méret miatt figyelmeztetés is lehet.

Mátrixok indexelése

```
m=matrix(sample(1:100,30,replace=T),nrow=5)
```

Előbb 30 véletlen számot sorsol 1 és 100 között visszatevéssel, majd ezekből 5 soros mátrixot készít. $30/5=6$, tehát a mátrix 5×6 -os lesz.

```
m
```

Kiírja/megjeleníti az m mátrix aktuális tartalmát.

```
m[13]
```

Egyetlen számmal indexelve a mátrixot R oszloponként "kilapítva" számozza. Az 5×6 -os m-ben a 13. elem a 3. sor, 3. oszlop helyén van.

```
m[2,5]
```

A 2. sor 5. oszlopában lévő egyetlen elemet választja ki. Mátrixnál az első index a sor, a második az oszlop.

```
m[1,4]
```

Az 1. sor 4. oszlopának elemét adja vissza.

```
m[4,4]
```

A 4. sor 4. oszlopának elemét adja vissza.

```
m[c(1,2),c(1,2)]
```

Az 1-2. sorokat ÉS az 1-2. oszlopokat választja ki. Az eredmény egy 2×2 -es részmátrix.

```
m[c(2,4),c(1,3,5)]
```

A 2. és 4. sort, valamint az 1., 3. és 5. oszlopot választja ki. Az eredmény 2×3 -as részmátrix.

```
m[-1,4]
```

A 4. oszlopot kéri, de az 1. sort kihagyja. Mivel egyetlen oszlop marad, R ezt általában vektorként adja vissza.

```
m[-c(1,2),c(4,5)]
```

Kihagyja az 1. és 2. sort, és csak a 4. és 5. oszlopot tartja meg. Így a 3-5. sorokból és 4-5. oszlopokból kapunk részmátrixot.

```
m[,2]
```

Az üres sorindex azt jelenti: minden sor. Tehát a teljes 2. oszlopot adja vissza.

```
m[3,]
```

Az üres oszlopindex azt jelenti: minden oszlop. Tehát a teljes 3. sort adja vissza.

Sorok hozzáadása és apply()

```
m2=rbind(m,c(1,3,5,7,9,11))
```

Az rbind() sorokat köt össze. Az m alá hozzáad egy új, 6 elemű sort. Mivel m 6 oszlopos, a hossz pontosan illeszkedik.

```
apply(m,1,mean)
```

Az apply() ugyanazt a függvényt alkalmazza a mátrix részeire. A 1 azt jelenti: soronként. Ez minden sor átlagát kiszámolja.

```
apply(m,2,mean)
```

A 2 azt jelenti: oszloponként. Ez minden oszlop átlagát kiszámolja.

```
apply(m,2,sd)
```

Minden oszlop szórását (sd) számolja ki.

```
apply(m,1,sum)
```

Minden sor elemeit összeadja, tehát sorösszegeket ad.

```
apply(m,2,mean)
```

A 2 azt jelenti: oszloponként. Ez minden oszlop átlagát kiszámolja.

```
m3=rbind(m,apply(m,2,mean))
```

Előbb kiszámolja minden oszlop átlagát, majd ezt az átlagvektort új utolsó sorként az m alá fűzi. Így az utolsó sor az oszlopátlagokat tartalmazza.

```
m3
```

Megjeleníti az m3 mátrixot, vagyis az eredeti m-et plusz az utolsó, oszlopátlagokat tartalmazó sort.